BAYESIAN MODELING OF COMPLEXITY

Markov Chain Monte Carlo (MCMC)

Meysam Goodarzi
Thursday 5th March, 2026

Hertie School of Public Governance

Agenda

1 Recap & Today

2 MCMC Foundations

- Markov Chains
- Monte Carlo

3 Metropolis-Hastings

4 MCMC Diagnostics

- Trace Plots
- R-hat Statistic
- Effective Sample Size
- Warmup and Convergence
- Divergences
- Practical Workflow

5 Wrap-up & Next

1 Recap & Today

2 MCMC Foundations

3 Metropolis-Hastings

4 MCMC Diagnostics

5 Wrap-up & Next

Today's Focus: MCMC in Practice

Learning objectives:

1. Understand **what** MCMC does (30% theory)
2. Know **when** your sampler has failed (70% diagnostics)

Reality check:

- You'll never code NUTS (No U-Turn Sampler) yourself
- PyMC handles sampling automatically
- **But you MUST diagnose problems when they occur**

Today's structure:

- Core concepts: Why MCMC? Markov chains, Metropolis-Hastings
- Practical diagnostics: Trace plots, R-hat, ESS, convergence
- Live coding: Simple MH implementation + PyMC workflow

The Problem: Computing Posteriors

Bayesian inference requires computing:

$$P(\theta|\mathbf{D}) = \frac{P(\mathbf{D}|\theta)P(\theta)}{P(\mathbf{D})}$$

The challenge:

- Denominator requires integrating over all possible θ:

$$P(\mathbf{D}) = \int P(\mathbf{D}|\theta)P(\theta)\, d\theta$$

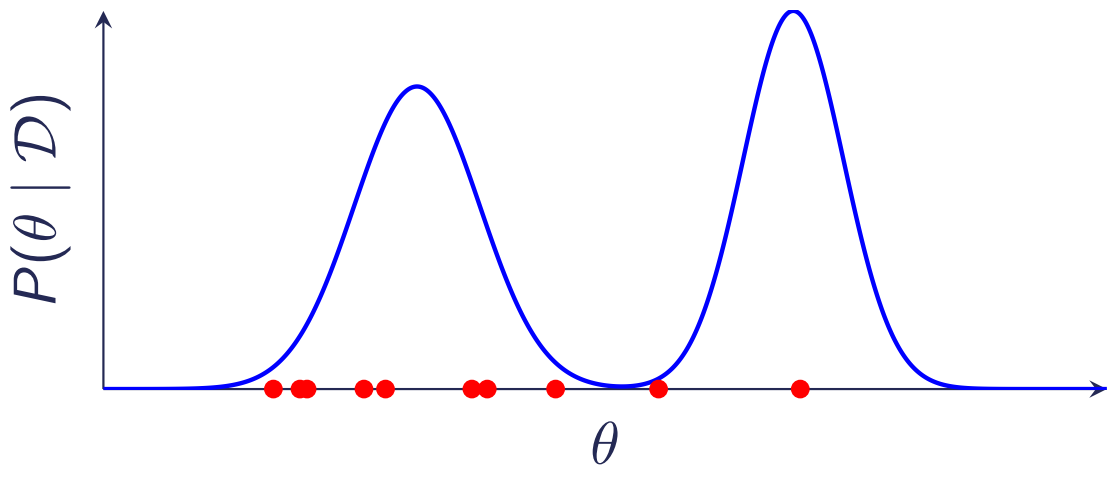
- Analytically tractable only for conjugate prior-likelihood pairs
- Most real models $\rightarrow$ **no closed-form solution**

Examples of non-conjugate models:

- Logistic regression with non-uniform priors
- Hierarchical models with multiple levels
- Agent-based model calibration

Why Not Just Sample Randomly?

Naive approach: Sample θ uniformly, compute posterior at each point



Problems:

- Wastes samples in low-probability regions
- Inefficient in high dimensions (curse of dimensionality)
- Needs exponentially more samples as dimensions increase

$\rightarrow$ **Solution:** Sample *smartly* using MCMC

1 Recap & Today

2 MCMC Foundations

- Markov Chains
- Monte Carlo

3 Metropolis-Hastings

4 MCMC Diagnostics

5 Wrap-up & Next

The Core Idea of MCMC

MCMC = Markov Chain + Monte Carlo

Key insight:

- Instead of sampling independently, build a **Markov chain**
- Design the chain so its **stationary distribution** $= P(\theta|\mathcal{D})$
- After "burn-in," samples approximate the posterior

Markov Chain:

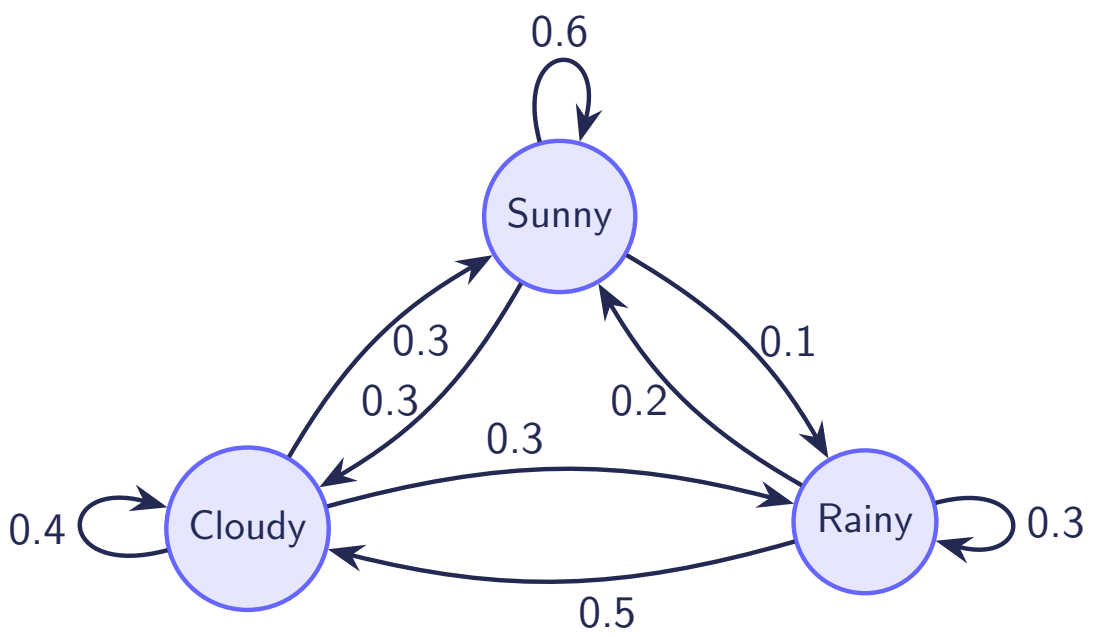
- Sequence of states
- Next state depends only on current state (memoryless)
- Transition probabilities $q(\theta'|\theta)$

Monte Carlo:

- Use random sampling
- Approximate expectations
- Law of large numbers guarantees convergence

Markov Chain: Weather Example

A Markov chain: A model of states connected by transition probabilities



Key properties:

- **Memoryless:** Tomorrow's weather depends only on today
- **Irreducible:** Can reach any state from any other state

Stationary Distribution

Question: If we run this chain forever, what fraction of time do we spend in each state?

Stationary distribution $\pi(\theta)$ satisfies:

$$\pi(\theta') = \sum_{\theta} q(\theta'|\theta)\pi(\theta)$$

For the weather example:

$$\begin{aligned} P(\text{Sunny}) &= 0.6P(\text{Sunny}) + 0.3P(\text{Cloudy}) + 0.2P(\text{Rainy}) \\ P(\text{Cloudy}) &= 0.3P(\text{Sunny}) + 0.4P(\text{Cloudy}) + 0.5P(\text{Rainy}) \\ P(\text{Rainy}) &= 0.1P(\text{Sunny}) + 0.3P(\text{Cloudy}) + 0.3P(\text{Rainy}) \end{aligned}$$

Solution:

$$P(\text{Sunny}) = 0.38, \quad P(\text{Cloudy}) = 0.40, \quad P(\text{Rainy}) = 0.22$$

MCMC goal: Design a chain where $\pi(\theta) = P(\theta|\mathcal{D})$

Monte Carlo Integration

Goal: Estimate quantities that are hard to compute analytically

Example:

$$I = \int_0^1 e^{-x^2}\, dx \quad \text{(no closed form!)}$$

Monte Carlo approximation:

$$I \approx \frac{1}{N} \sum_{i=1}^{N} e^{-U_i^2}, \quad U_i \sim \text{Uniform}(0, 1)$$

Law of Large Numbers: As $N \to \infty$, this average converges to the true value

In Bayesian inference:

$$\mathbb{E}[f(\theta)|\mathcal{D}] = \int f(\theta) P(\theta|\mathcal{D}) d\theta \approx \frac{1}{N} \sum_{i=1}^{N} f(\theta_i)$$

where θ_i are samples from the posterior

1 Recap & Today

2 MCMC Foundations

3 Metropolis-Hastings

4 MCMC Diagnostics

5 Wrap-up & Next

The Metropolis-Hastings Algorithm

Goal: Design a Markov chain with stationary distribution $P(\theta|\mathcal{D})$

Algorithm: Starting from $\theta^{(0)}$, repeat for $t = 1, 2, \ldots$

1. **Propose** a new state: $\theta' \sim q(\cdot|\theta^{(t)})$ (e.g., $\theta' = \theta^{(t)} + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$)
2. **Compute** acceptance ratio:

$$\alpha = \min\left(1, \frac{P(\theta'|\mathcal{D})\, q(\theta^{(t)}|\theta')}{P(\theta^{(t)}|\mathcal{D})\, q(\theta'|\theta^{(t)})}\right)$$

3. **Accept or reject:**
   - With probability α: set $\theta^{(t+1)} = \theta'$
   - With probability $1 - \alpha$: set $\theta^{(t+1)} = \theta^{(t)}$ (stay put)

Note: If q is symmetric, e.g., random walk, acceptance simplifies to:

$$\alpha = \min\left(1, \frac{P(\theta'|\mathcal{D})}{P(\theta^{(t)}|\mathcal{D})}\right)$$

Why Does Metropolis-Hastings Work?

Intuition:

- If θ' has **higher** posterior probability $\rightarrow$ always accept
- If θ' has **lower** posterior probability $\rightarrow$ sometimes accept
- The chain spends more time in high-probability regions

Detailed Balance

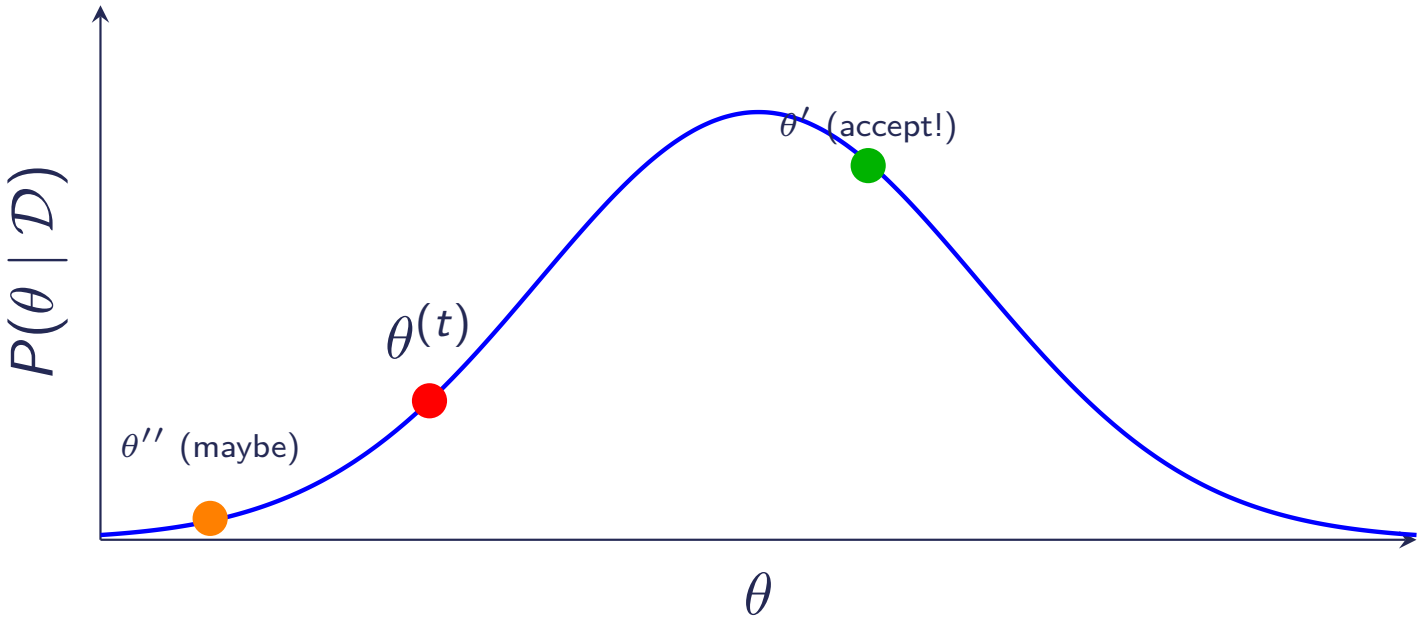
$$P(\theta|\mathcal{D}) \cdot P(\theta \rightarrow \theta') = P(\theta'|\mathcal{D}) \cdot P(\theta' \rightarrow \theta)$$

This ensures that the chain's stationary distribution is $P(\theta|\mathcal{D})$

Design choices:

- **Proposal distribution q:** Random walk is simplest, but not always efficient
- **Step size σ:** Too small $\rightarrow$ slow exploration; too large $\rightarrow$ many rejections

MH Algorithm: Visual Intuition



Proposals:

- Green dot: Higher probability $\rightarrow$ always accept
- Orange dot: Lower probability $\rightarrow$ accept with probability $P(\theta'')/P(\theta^{(t)})$

Beyond Metropolis-Hastings

MH is the foundation, but modern samplers are more sophisticated:

- **Gibbs Sampling:** Update one parameter at a time (useful for conjugate substructures)
- **Hamiltonian Monte Carlo (HMC):** Uses gradient information for more efficient proposals
- **No U-Turn Sampler (NUTS):** Automatically tunes HMC (PyMC's default)

What you need to know:

- All MCMC algorithms build a Markov chain
- They differ in how they propose new states
- PyMC chooses the best algorithm automatically
- **Your job:** Check if the sampler worked!

1 Recap & Today

2 MCMC Foundations

3 Metropolis-Hastings

4 MCMC Diagnostics

- Trace Plots
- R-hat Statistic
- Effective Sample Size
- Warmup and Convergence
- Divergences
- Practical Workflow

The Critical Question: Did It Work?

MCMC is an approximation:

- We run chains for a finite number of iterations
- We discard early samples (burn-in / warmup)
- We hope the remaining samples represent $P(\theta|\mathcal{D})$

Things that can go wrong:

- Chain hasn't converged yet (needs more iterations)
- Chain is stuck in one region (poor mixing)
- Multiple chains end up in different places (multimodal posterior)
- Sampler can't handle the geometry (e.g., strong correlations)

This section: Tools to diagnose these problems

Trace Plots: Visual Inspection

Trace plot: Shows the value of $\theta^{(t)}$ over iterations

Good trace plot:

- Looks like "fuzzy caterpillar"
- No trends or patterns
- Explores full range
- Multiple chains overlap

Bad trace plot:

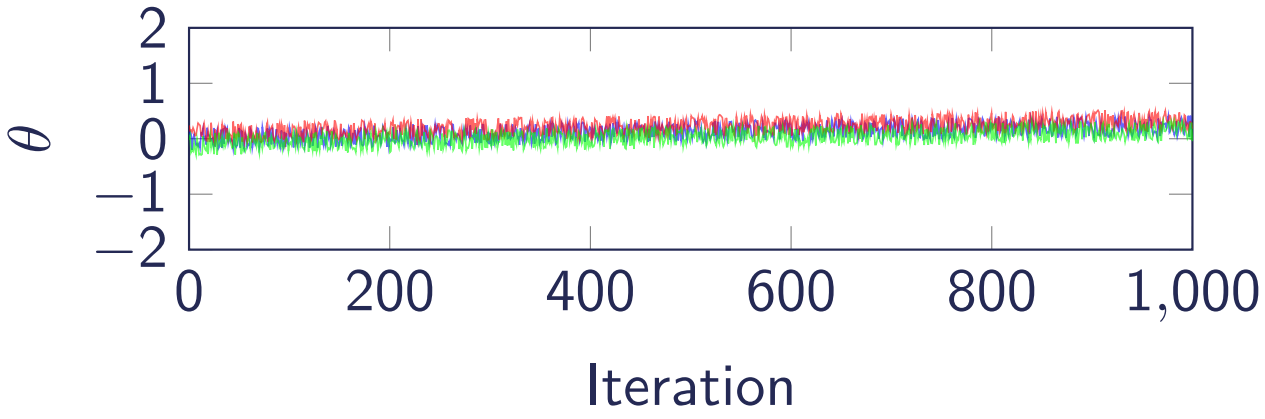
- Shows trends (not converged)
- Stuck in one region
- Chains don't overlap
- Sudden jumps or gaps

In practice:

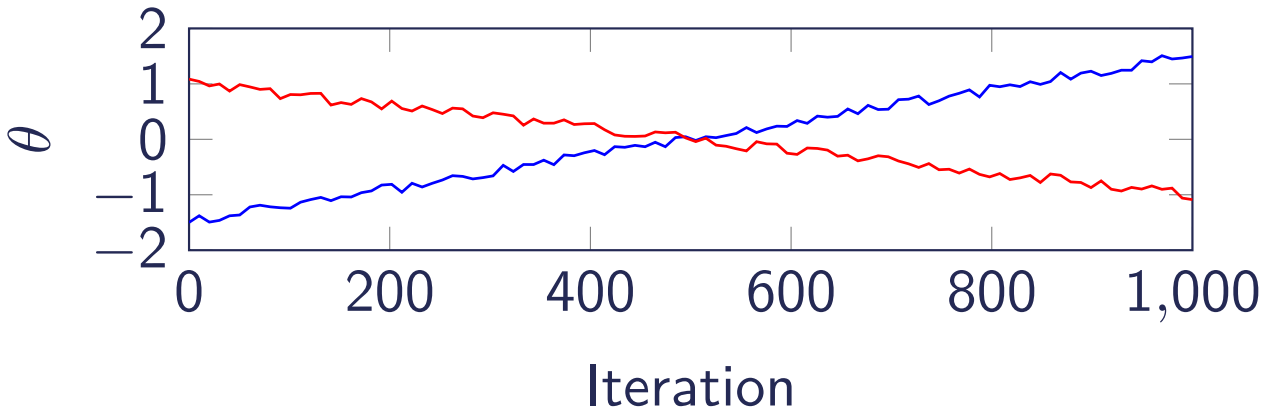
- Always run multiple chains (e.g., 4 chains)
- Plot all chains on the same axis
- Use: `az.plot_trace(inference_data)`

Trace Plot Examples

Good convergence:



Poor convergence:



R-hat: The Gold Standard

R-hat (Gelman-Rubin statistic): Compares variation *between* chains to variation *within* chains

$$\hat{R} = \sqrt{\frac{\text{Var}_{\text{between}}}{\text{Var}_{\text{within}}}}$$

Interpretation:

- $\hat{R} \approx 1$: Chains have converged
- $\hat{R} > 1.01$: **Warning** - chains disagree, run longer
- $\hat{R} > 1.1$: **Serious Problem** - do not trust results

Why it works:

- If chains haven't converged, they 'll be in different regions
- Between-chain variance will be large
- If converged, all chains explore the same distribution

Rule of thumb: $\hat{R} < 1.01$ for all parameters

Effective Sample Size (ESS)

Problem: MCMC samples are *correlated*

- $\theta^{(t)}$ depends on $\theta^{(t-1)}$ (it's a Markov chain!)
- 1000 MCMC samples $\neq$ 1000 independent samples

Effective Sample Size (ESS):

$$\text{ESS} \approx \frac{N}{1 + 2\sum_{k=1}^{\infty} \rho_k}$$

where ρ_k is the autocorrelation at lag k

Interpretation:

- ESS = effective number of independent samples
- Higher is better
- Rule of thumb: ESS $>$ 400 per chain (or ESS $>$ 1600 total for 4 chains)

If ESS is low: Run more iterations or improve sampler efficiency

Warmup (Burn-in)

Problem: MCMC chains need time to reach the stationary distribution

- Starting point $\theta^{(0)}$ is often arbitrary
- Early samples are biased toward the starting point
- Solution: Discard the first N_{warmup} samples

In practice:

- PyMC default: 1000 warm-up + 1000 sampling iterations
- The warm-up phase also tunes the sampler parameters (e.g., step size)
- Check trace plots: do chains stabilize after warm-up?

How much warm-up is enough?

- No universal answer
- Check: R-hat < 1.01 and trace plots look good
- If not, increase both warmup and sampling iterations

Divergences: When the Sampler Fails

Divergence: Sampler encounters numerical instability

- Specific to HMC/NUTS (PyMC's default sampler)
- Indicates the sampler couldn't follow the posterior accurately
- Often occurs in regions of high curvature

What to do if you see divergences:

1. **Reparameterize:** Change how you express the model
2. **Increase target acceptance:**
   `pm.sample(target_accept=0.95)`
3. **Use stronger priors:** Regularize extreme parameter values

Warning signs:

- PyMC will print: "There were X divergences after tuning"
- Even 1 divergence is worth investigating
- Many divergences $\rightarrow$ results may be unreliable

MCMC Diagnostic Workflow

After running MCMC, always check (in this order):

1. **Warnings:** Did PyMC report divergences or other issues?
2. **Trace plots:** Visual inspection using `az.plot_trace()`
   - Do chains look like "fuzzy caterpillars"?
   - Do all chains overlap?
3. **R-hat:** Check `az.summary()`
   - Is $\hat{R} < 1.01$ for all parameters?
4. **ESS:** Also in `az.summary()`
   - Is ESS > 400 per chain?

If any check fails:

- Increase iterations: `pm.sample(draws=2000, tune=2000)`
- Try different initialization: `pm.sample(init='adapt_diag')`
- Simplify the model or add priors

1 Recap & Today

2 MCMC Foundations

3 Metropolis-Hastings

4 MCMC Diagnostics

5 Wrap-up & Next

Wrap-up

Concepts:

- MCMC samples from posteriors by building Markov chains
- MH: propose, accept/reject based on posterior ratio
- Modern samplers (NUTS) follow the same logic (with higher efficiency)

Diagnostics (this is the critical skill):

- **Trace plots:** Visual check for convergence
- **R-hat** < 1.01**:** Chains have converged
- **ESS** > 400**:** Enough effective samples
- **No divergences:** Sampler is working correctly

Practical remarks:

- Always run multiple chains (default: 4)
- Always check diagnostics before trusting results
- When in doubt, run more iterations

Next Week: Bayesian Linear Models

Now that we can sample from posteriors:

- Apply MCMC to regression problems
- Compare Bayesian vs. frequentist inference
- Use PyMC for real-world modeling

Preparation:

- Review linear regression (frequentist version)
- Install PyMC and ArviZ: `pip install pymc arviz`
- Review today's live coding notebooks on Moodle

Questions?

Next: Bayesian Linear Models!